INDIVIDUAL IN-DEPTH STUDY

Exploring Django through a Standalone CRUD Micro-Prototype

Utveckling av webbapplikationer (C1UT1B)
Högskolan i Borås

Rasmus Serrestam

Repository link for micro-artifact:
https://github.com/Rasmus-04/Django-Individual-in-Depth-study

Abstract

This study explores Django as a web framework by implementing a small standalone web prototype for creating, viewing, editing and deleting posts. The purpose is to examine Djangos architectural approach and compare its model, routing, form handling and developer experience with ASP.NET Core MVC, which was used in Laboratory Assignment 2. The method combines a review of official framework documentation with an inspection of the implemented prototype and a feature-based comparison. The prototype uses a Django model for post data, function-based views for request handling, URL patterns for routing, a ModelForm for input handling, Django templates for HTML rendering and SQLite as its configured database. The implementation demonstrates a path from an HTTP request to database operations and a rendered response.

Django provides integrated components for common web-development tasks, which reduces the amount of infrastructure that must be built manually. ASP.NET Core MVC offers a more explicitly separated controller, view and model structure within the .NET ecosystem and can be a natural fit for applications already using C# and Entity Framework Core. This study does not include controlled performance benchmarks or a usability study, so conclusions about speed are not made. The comparison instead focuses on architecture and the development workflow observed while building the prototype. The findings suggest that Django is effective for quickly building database-backed applications, while the better choice depends on the developers language, existing system, architectural needs and project constraints.

1. Introduction

1.1 Background and problem statement

Web frameworks provide reusable structures for handling HTTP requests, routing, data persistence, validation and the generation of web pages. Without a framework, a developer would need to implement or assemble many of these mechanisms independently. Frameworks differ in how much functionality they include by default and how they organise an application.

In Laboratory Assignment 2, I worked with ASP.NET Core MVC, C# and Entity Framework Core to develop a database-driven web application. For this individual study, I selected Django, a Python web framework, and built a standalone micro-prototype. I chose a simple post-management application so that I could focus on learning Django and exploring its main features instead of spending too much time defining a new application domain. The prototype supports the four basic CRUD operations: create, read, update and delete.

The main problem investigated in this report is how Djangos integrated architecture affects the implementation of a small data-driven web application, and what trade-offs become visible when it is compared with ASP.NET Core MVC.

1.2 Purpose and research question

The purpose of this study is to learn how Django works in practice and compare it with ASP.NET Core MVC, which I used in Laboratory Assignment 2. I focus on how the two frameworks handle application structure, routing, forms, state management and the overall development process. By implementing a small application in Django, I aim to understand the advantages and disadvantages of the framework and how it differs from the approach used in Lab 2.

Research question: What are the architectural advantages and disadvantages of Django, and how do its component model, state management, and developer experience differ from ASP.NET Core MVC as applied in Laboratory Assignment 2?

1.3 Target audience and significance

This study is aimed at students and developers who want to learn more about web frameworks and understand the differences between them. By comparing Django with ASP.NET Core MVC, the study shows how different frameworks approach common web development tasks. Understanding these differences can help developers choose a framework that fits their experience, project requirements and preferred programming language.

1.4 Literature review

Django is a high-level Python web framework that provides built-in functionality for common web development tasks. Its documentation describes models for defining application data, URL configurations for routing requests, views for handling application logic and templates for generating HTML pages (Django Software Foundation, n.d.-b; n.d.-c; n.d.-d). These features are relevant to this study because they are used in my prototype to implement the different CRUD operations.

Another feature of Django is ModelForm, which makes it possible to create forms based on existing models. This reduces the need to define the same fields separately and provides a way to handle form validation (Django Software Foundation, n.d.-a). In my implementation, I use ModelForm to handle the fields for creating and editing posts. This allows me to examine how Django connects its data models with form handling.

Research by Torres and Hora (2026) examines the ecosystem of reusable Django applications. Their study analyses 487 applications and investigates aspects such as application activity, testing and compatibility. Their findings show that reusable applications can provide useful functionality, but that compatibility and maintenance are also important considerations. This is relevant to the present study because it highlights that the advantages of a framework depend not only on its built-in features, but also on the wider ecosystem available to developers.

ASP.NET Core MVC uses the Model-View-Controller pattern to separate application data, request handling and presentation (Microsoft, n.d.-a). In addition, Entity Framework Core provides functionality for working with databases through .NET objects (Microsoft, n.d.-b). These technologies were used in Laboratory Assignment 2 and provide a basis for comparing the two frameworks.

The documentation and research provide a theoretical basis for understanding the frameworks and their features, but they do not establish which framework is best for every type of application. Therefore, I compare this information with my own implementation and experience from Lab 2. This makes it possible to discuss the practical differences between Django and ASP.NET Core MVC without making unsupported claims about performance or general developer productivity.

2. Methods

2.1 Study object and scope

The study object is a standalone Django project named Individual in-depth study. The project uses Django, Python, SQLite and Bootstrap for basic styling. The main application is named core, and its central entity is Post. The model contains the fields user_id, text, status, created_date and last_edited_date.

The prototype supports the four basic CRUD operations: creating, viewing, editing and deleting posts. It uses Django models to define the data, views to handle requests, URL configurations for routing, a ModelForm for form handling and HTML templates to generate the pages.

The project follows the standalone micro-prototype track. The scope is limited to CRUD functionality for one entity, rather than a complete web application. Features such as full authentication, authorisation and deployment are outside the scope of this study. This makes it possible to focus on Django's main features and how they are used to build a small database-driven application.

2.2 Literature search procedure

The literature search focused on finding sources that explain the main features and architecture of Django and ASP.NET Core MVC. I mainly used the official Django documentation and Microsoft documentation for ASP.NET Core MVC and Entity Framework Core. These sources were selected because they describe how the frameworks work and provide information about the components used in my implementations.

I focused on documentation about models, URL routing, templates and ModelForm for Django, as well as MVC architecture and database access for ASP.NET Core. I also included the research article by Torres and Hora (2026), which examines reusable applications in the Django ecosystem. I selected this article because it provides a research-based perspective on the benefits and maintenance challenges of reusable software.

The sources were selected based on their relevance to the research question, their connection to the framework features examined in the study, and the credibility of the publisher or authors. Official documentation was used to understand how the frameworks are intended to work, while the research article provided an additional academic perspective. Sources that did not directly contribute to understanding the architecture, component model or development process were not prioritised.

The selected literature was then compared with my Django implementation and my experience from Laboratory Assignment 2. This allowed me to connect the theoretical descriptions of the frameworks with practical observations from the two projects.

2.3 Implementation and evaluation protocol

I developed the prototype by working with the main parts of a Django project, including the model, form, views, URL configuration and HTML templates. I used these components to implement the CRUD operations for posts. During development, I examined how requests are routed, how data is retrieved from and saved to the database, and how forms are used to create and update records.

After implementing the prototype, I compared Django with the ASP.NET Core MVC approach used in Laboratory Assignment 2. The comparison focused on application structure, routing, data models, database operations, form validation, state management and the overall development experience. I considered how the code is organised, how much functionality is provided by the frameworks and how the different tasks are implemented.

The evaluation is based on the structure of the code, the features used in the prototype and my experience of working with both frameworks. I did not perform controlled performance tests or user experiments. Therefore, the study does not establish which framework is faster or more productive in general. Instead, it examines the architectural differences and practical trade-offs that became clear while implementing the prototype and comparing it with Laboratory Assignment 2.

3. Results

3.1 Micro-artifact overview

The micro-artifact is a small web application built with Django. It allows users to create, view, edit and delete posts. The purpose of the implementation is to demonstrate how Django handles common web development tasks, such as routing, form handling, database operations and the generation of HTML pages.

The project is divided into a Django project named config and an application named core. The main data model is Post, which contains the fields user_id, text, status, created_date and last_edited_date. The database is configured to use SQLite, while Bootstrap is used for the user interface.

The implementation uses function based views to handle requests. The URL configuration maps each URL to the corresponding view. For example, the route for creating a post is connected to post_create, while the routes for editing and deleting posts include the post ID. The views use the Django ORM to retrieve, create, update and delete records in the database.

```
def post_list(request):
    posts = Post.objects.all().order_by("-created_date")

    return render(
        request,
        "core/post_list.html",
        {"posts": posts}
    )
```

This view retrieves all posts from the database and sorts them by creation date, with the newest posts first. It then passes the posts to the HTML template through the posts context variable. This example shows how Django connects database access, request handling and presentation.

Djangos ModelForm is used for creating and editing posts. It connects the form to the Post model and handles form validation. When a valid form is submitted, the data is saved to the database. When a post is edited, the last_edited_date field is updated. The templates then display the data returned by the views.

The main request flow can be described as follows:

1. The user opens a URL or submits a form.
2. Django matches the URL to a view.
3. The view retrieves or updates data through the model and database.

4. The view returns a rendered HTML template or redirects the user to another page.

The following diagram shows how the main components of the Django prototype work together when a user interacts with the application.

Figure 1. Architecture and request flow of the Django micro-prototype.

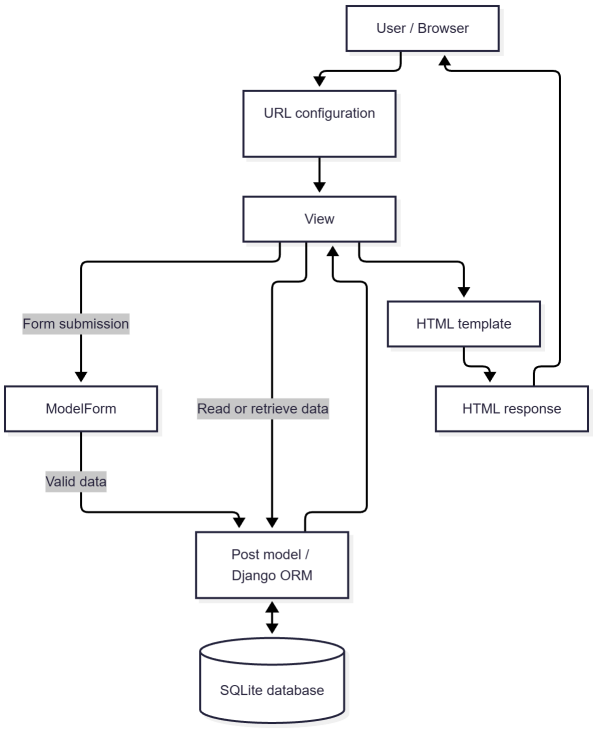


Figure 1 illustrates how the main components of the Django prototype work together. The URL configuration directs requests to a view, which handles the application logic and interacts with the model and database when needed. The view then returns a rendered HTML page or redirects the user.

This structure separates the main responsibilities of the application. The URL configuration handles routing, the views handle requests, the model represents the post data, the ModelForm handles form input, and the templates generate the HTML pages. These components work together to implement the CRUD functionality.

3.2 Comparative technical analysis

The comparison is based on the Django prototype and the ASP.NET Core MVC application developed in Laboratory Assignment 2. Both projects include functionality for managing posts, which provides a useful basis for comparison. However, the projects do not have exactly the same scope. Lab 2 includes user accounts, comments, friend requests and

additional post functionality, while the Django prototype focuses on the basic CRUD operations for one entity.

3.2.1 Component model and application structure

Django separates the application into models, views, URL configurations, forms and templates. The model defines the data structure, the URL configuration maps requests to views, and the views contain the logic for handling requests. Templates are used to generate the HTML shown to the user.

In Lab 2, ASP.NET Core MVC uses controllers to handle requests, models to represent application data and Razor views to generate the user interface. Database operations are handled through Entity Framework Core and the application's database context. The separation between controllers and views is more explicit in the MVC structure, while Django provides additional integrated components, such as its ORM and ModelForm.

Both frameworks separate application logic from presentation, but they organise some responsibilities differently. Django's ModelForm connects the model to form handling, whereas the Lab 2 implementation uses the ASP.NET Core model-binding and validation mechanisms in its controller actions and Razor views.

3.2.2 Routing and request handling

In Django, routes are defined in core/urls.py using path(). Each route points directly to a view function. For example, the edit and delete routes include an integer post ID, which the view uses to find the corresponding record.

In ASP.NET Core MVC, routing follows the controller and action structure. The PostsController contains actions such as Index, Create, Edit and Delete. These actions handle incoming requests and return views or redirects. The controller can also use dependency injection to access the application's database context.

The main difference is how the request-handling logic is organised. Djangos prototype connects URL patterns directly to individual view functions, while Lab 2 groups related actions in a controller class.

3.2.3 Data models, forms and database operations

Both projects use an object-relational mapping approach to work with relational databases. Django uses its built-in ORM and a Post model, while Lab 2 uses Entity Framework Core and a Post model connected to an ApplicationDbContext.

The Django model defines the post fields and validation-related constraints. The PostForm uses the model to generate a form for the user_id, text and status fields. In the create and update views, the form is validated before its data is saved. Django also provides get_object_or_404() for retrieving a post or returning a not-found response if it does not exist.

In Lab 2, the controller uses Entity Framework Core to query and update posts. For example, the Index action retrieves posts and their associated users, while the Create, Edit and Delete actions handle changes to the database. The Razor views use model binding and validation helpers to display forms and validation messages.

Although both frameworks simplify database operations, the implementation in Lab 2 includes relationships between posts and user accounts. In the Django prototype, user_id is stored as a character field rather than as a relationship to an authenticated user. This makes the Django data model simpler, but it does not provide the same user-related functionality.

3.2.4 State management

Both applications use a server-rendered request-and-response approach rather than a client-side single-page application. The browser sends a request, the server processes it and the server returns an HTML response or a redirect. The post data is stored in the database, so changes remain available after a request has finished.

In Django, a view passes data to a template through the context dictionary. For example, post_list passes the posts to the template using the posts context variable. When a form is submitted, the view validates the submitted data and saves it if it is valid.

In Lab 2, controller actions pass models or collections to Razor views. Entity Framework Core handles database operations, and redirects are used after successful create, edit and delete operations. ASP.NET Core also has session and other state-management facilities, but the compared post-management functionality mainly relies on database persistence and request data.

The main difference observed in these implementations is therefore not that one framework uses persistent state and the other does not. Both persist post data in a database. Instead, they differ in how data is passed between request handlers and views and how the frameworks organise form handling and database access.

3.2.5 Developer experience

The Django prototype uses a relatively small number of files to implement the basic CRUD operations. The model, form, views, routes and templates each have a clear role. ModelForm

reduces the need to define the same form fields separately from the model, and the ORM makes it possible to work with database records through Python objects.

Lab 2 uses more components, including controllers, Razor views, an application database context, Entity Framework Core and ASP.NET Core Identity. This adds complexity to the project, but it also supports features that are outside the scope of the Django prototype, such as authenticated users and relationships between posts and users.

Based on these implementations, Django offers a direct way to build a small database-driven prototype, while ASP.NET Core MVC provides a structure that supports a larger application with additional services and security requirements. This is an observation from the two projects rather than a general measurement of developer productivity.

3.2.6 Comparative summary

Table 1 summarises the main technical differences observed in the two implementations.

Area	Django prototype	ASP.NET Core MVC in Lab 2
Language	Python	C#
Application structure	Models, function-based views, URL configuration, forms and templates	Models, controllers, Razor views and supporting services
Routing	Explicit URL patterns mapped to view functions	Controller and action-based routing
Database access	Built-in Django ORM	Entity Framework Core
Form handling	ModelForm connected to the Post model	Model binding, validation and Razor form helpers
Data persistence	SQLite	SQLite through Entity Framework Core
Post and user relationship	user_id stored as a character field	Post linked to an application user through a foreign key and navigation property
State handling	Request data, template context and database persistence	Request data, controller-to-view models and database persistence

Authentication and authorisation	Not implemented as part of the prototype	ASP.NET Core Identity and authorisation are used
Scope	Basic CRUD for one entity	Larger social application with posts, comments, friend requests and user accounts
Observed development experience	Integrated components make basic CRUD implementation direct	More setup and components, but supports more complex application requirements
Performance	Not benchmarked	Not benchmarked

The comparison shows that both frameworks provide the main mechanisms needed to build a database-driven web application. Django integrates model, form and template functionality into a compact structure, while ASP.NET Core MVC separates request handling, presentation and database access through controllers, Razor views and Entity Framework Core. The additional functionality in Lab 2 also means that the difference in project size cannot be attributed to the frameworks alone.

Performance was not measured in this study. No controlled benchmarks were conducted, so the results cannot establish whether Django or ASP.NET Core MVC performs better at runtime. The comparison instead focuses on the architecture, features and development process observed in the two implementations.

4. Discussion & Conclusion

4.1 Interpretation and reflection

The results show that Django provides an integrated structure for building a small database-driven web application. In my prototype, models, views, URL configurations, forms and templates each have a specific role. These components work together to implement the CRUD operations without requiring a large amount of additional infrastructure. One example is ModelForm, which connects the form fields to the Post model and provides form validation. This made it possible to implement form handling without defining all the fields separately.

Compared with ASP.NET Core MVC in Laboratory Assignment 2, Django organises some parts of the application differently. Django connects URL patterns directly to view functions, while ASP.NET Core MVC groups related request-handling actions in controllers and uses Razor views for the presentation. Both frameworks separate application logic from presentation, but Django provides more integrated functionality for tasks such as form handling and database access through its ORM.

From my experience, Django was a straightforward way to build the basic CRUD functionality. Its built-in components made it easier to understand how requests, database operations and templates work together. ASP.NET Core MVC required working with more components in Lab 2, including controllers, Razor views, Entity Framework Core and ASP.NET Core Identity. However, Lab 2 also included more functionality, so the difference in complexity cannot be explained by the frameworks alone.

The comparison also shows that both frameworks support a server-rendered application where data is stored in a database. The main difference in state handling is how data is passed between request handlers and views, rather than whether the frameworks support persistent data. Django uses template context to pass data to templates, while ASP.NET Core MVC passes models or collections from controller actions to Razor views.

These observations are consistent with the official documentation for both frameworks. The Django documentation describes how models, URL configurations and templates have separate roles in an application, while ModelForm connects model fields with form handling (Django Software Foundation, n.d.-a; n.d.-b; n.d.-c; n.d.-d). This can also be seen in my prototype, where these components work together to implement the CRUD operations. Microsoft documentation describes a similar separation of responsibilities through controllers, views and Entity Framework Core (Microsoft, n.d.-a; n.d.-b). The comparison therefore shows how both frameworks support the same general tasks while organising some of their components differently.

4.2 Limitations and developer experience

There are some limitations to this study. First, the Django prototype is smaller than the application developed in Lab 2. It only implements CRUD operations for one entity and does not include the same authentication, authorisation and user-related functionality. This means that the comparison does not cover all features of either framework.

Second, the evaluation is based on the code, official documentation and my own experience of implementing the prototype and working with ASP.NET Core MVC. I did not conduct controlled performance tests or user experiments. Therefore, I cannot draw conclusions about which framework is faster or which one provides better productivity in general. The results instead show how the frameworks organise common web development tasks in these two implementations.

Another limitation is that the Django prototype stores user_id as a character field rather than linking each post to an authenticated user through a database relationship. This keeps the model simple, but it also means that the prototype does not provide the same user-related functionality as Lab 2. A more complete application would require additional work on authentication, authorisation and relationships between users and posts.

The implementation also helped me understand the advantages and trade-offs of using a framework with integrated components. Django provides a direct way to connect models, forms and views, which is useful when developing a small application. At the same time, the developer still needs to understand how these components work together and how the application should be structured. ASP.NET Core MVC uses a more explicitly separated controller, view and model structure, which can be useful when an application has more functionality and needs additional services.

Torres and Hora (2026) examined 487 reusable applications in the Django ecosystem and identified compatibility and maintenance as important considerations. Their findings provide a broader perspective on the trade-offs involved in using reusable software. My prototype uses Djangos built-in components rather than evaluating a collection of third-party applications, so the study does not directly evaluate the same situation. However, it highlights that having reusable functionality available is not the only consideration; compatibility and maintenance can also matter when developing and extending an application.

4.3 Conclusion

The purpose of this study was to explore Django through a standalone CRUD prototype and compare it with ASP.NET Core MVC as used in Laboratory Assignment 2. The results show that Djangos main architectural advantage is its integrated approach to common web development tasks. Its ORM, URL configuration, views, ModelForm and templates provide the components needed to build a database-driven application with a relatively small amount of code.

Compared with ASP.NET Core MVC, Django connects routing directly to views and provides integrated form handling through ModelForm. ASP.NET Core MVC separates request handling into controllers and presentation into Razor views, while Entity Framework Core handles database operations. Both frameworks support database persistence and server-rendered pages, but they organise their components differently.

The main disadvantage observed in my Django prototype is not a limitation of Django itself, but that the prototype lacks features such as authentication, authorisation and relationships between posts and users. These features would need to be implemented to provide functionality comparable to Lab 2. The study also does not establish which framework performs better because no performance benchmarks were conducted.

Overall, Django was a suitable choice for this study because it allowed me to explore the main parts of a web framework through a small practical implementation. The choice between Django and ASP.NET Core MVC depends on factors such as programming language, existing systems, application requirements and the developer's experience. This study helped me understand how the two frameworks approach similar tasks and the trade-offs involved in their different structures.

References

- Django Software Foundation (n.d.-a) ‘ModelForm’. *Django documentation*. Available at: https://docs.djangoproject.com/en/6.1/topics/forms/modelforms/ (Accessed: 26 October 2026).
- Django Software Foundation (n.d.-b) ‘Models’. *Django documentation*. Available at: https://docs.djangoproject.com/en/6.1/topics/db/models/ (Accessed: 26 October 2026).
- Django Software Foundation (n.d.-c) ‘The Django template language’. *Django documentation*. Available at: https://docs.djangoproject.com/en/6.1/ref/templates/language/ (Accessed: 26 October 2026).
- Django Software Foundation (n.d.-d) ‘URL dispatcher’. *Django documentation*. Available at: https://docs.djangoproject.com/en/6.1/topics/http/urls/ (Accessed: 26 October 2026).
- Microsoft (n.d.-a) ‘Overview of ASP.NET Core MVC’. *Microsoft Learn*. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview (Accessed: 26 October 2026).
- Microsoft (n.d.-b) ‘Entity Framework Core’. *Microsoft Learn*. Available at: https://learn.microsoft.com/en-us/ef/core/ (Accessed: 26 October 2026).
- Torres, R. and Hora, A. (2026) ‘Assessing reusable web applications: The Django ecosystem case’, *Software Quality Journal*, 34, article 11. Available at: https://doi.org/10.1007/s11219-026-09745-8.